

INFO145 - Diseño y Análisis de Algoritmos

2 - Análisis de un Algoritmo

Héctor Ferrada

académico

Instituto de Informática
Universidad Austral de Chile

Análisis de un Algoritmo

Sea el problema $\mathcal{P}$ de encontrar el valor mínimo en una secuencia de n números enteros.

Algoritmo $\mathcal{A}$ que determina el mínimo de la instancia de entrada.

Input: un arreglo de n enteros $A[1..n]$

Output: min , el elemento menor en A

```
minimo( $A$ ,  $n$ ){  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Suponga que las **Operaciones Primitivas** $\rightarrow$ todas las comparaciones:

Análisis de un Algoritmo

Sea el problema $\mathcal{P}$ de encontrar el valor mínimo en una secuencia de n números enteros.

Algoritmo $\mathcal{A}$ que determina el mínimo de la instancia de entrada.

Input: un arreglo de n enteros $A[1..n]$

Output: min , el elemento menor en A

```
minimo( $A$ ,  $n$ ){  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Suponga que las **Operaciones Primitivas** $\rightarrow$ **todas las comparaciones**:

- Comparaciones del for $\rightarrow n$ (desde $i = 2$ hasta que $i = n + 1$ y sale del ciclo)
- Comparaciones de la sentencia if $\rightarrow n - 1$ (ya que entra $n - 1$ veces en el ciclo)

Análisis de un Algoritmo

Sea el problema $\mathcal{P}$ de encontrar el valor mínimo en una secuencia de n números enteros.

Algoritmo $\mathcal{A}$ que determina el mínimo de la instancia de entrada.

Input: un arreglo de n enteros $A[1..n]$

Output: min , el elemento menor en A

```
minimo( $A$ ,  $n$ ){  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Suponga que las **Operaciones Primitivas** $\rightarrow$ **todas las comparaciones**:

- Comparaciones del for $\rightarrow n$ (desde $i = 2$ hasta que $i = n + 1$ y sale del ciclo)
 - Comparaciones de la sentencia if $\rightarrow n - 1$ (ya que entra $n - 1$ veces en el ciclo)
- $\therefore$ Costo $T_{\mathcal{A}}(n) = n + n - 1 = 2n - 1 \leq 2n = c \cdot n$, con $c > 0$ constante

Análisis de un Algoritmo

Sea el problema $\mathcal{P}$ de encontrar el valor mínimo en una secuencia de n números enteros.

Algoritmo $\mathcal{A}$ que determina el mínimo de la instancia de entrada.

Input: un arreglo de n enteros $A[1..n]$

Output: min , el elemento menor en A

```
minimo( $A$ ,  $n$ ){  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Suponga que las **Operaciones Primitivas** $\rightarrow$ todas las comparaciones:

- Comparaciones del for $\rightarrow n$ (desde $i = 2$ hasta que $i = n + 1$ y sale del ciclo)
- Comparaciones de la sentencia if $\rightarrow n - 1$ (ya que entra $n - 1$ veces en el ciclo)

$\therefore$ Costo $T_{\mathcal{A}}(n) = n + n - 1 = 2n - 1 \leq 2n = c \cdot n$, con $c > 0$ constante

$T_{\mathcal{A}}(n) \rightarrow O(n)$, o simplemente $T(n) = O(n) \rightarrow$ ¿Qué significa?

Análisis de un Algoritmo

- Sea $\mathcal{P}$ un problema bien definido en nuestro modelo de computo, como el de la slide anterior. $\mathcal{P}$: encontrar el elemento menor de un arreglo de enteros $A[1..n]$.

Definimos a $D_{\mathcal{P}}(n)$ como al **conjunto de todas las posibles entradas de tamaño n al problema $\mathcal{P}$** .

Para el problema $\mathcal{P}$, ¿Cuánto es $|D_{\mathcal{P}}(n)|$?, con $n = 12$.

Análisis de un Algoritmo

- Sea $\mathcal{P}$ un problema bien definido en nuestro modelo de computo, como el de la slide anterior. $\mathcal{P}$: encontrar el elemento menor de un arreglo de enteros $A[1..n]$.

Definimos a $D_{\mathcal{P}}(n)$ como al **conjunto de todas las posibles entradas de tamaño n al problema $\mathcal{P}$** .

Para el problema $\mathcal{P}$, ¿Cuánto es $|D_{\mathcal{P}}(n)|$?, con $n = 12$.

- Llamamos **Instancia $\mathcal{I} \in D_{\mathcal{P}}(n)$** a una entrada específica de tamaño n , y definimos $T_{\mathcal{A}, \mathcal{P}}(\mathcal{I})$ como el **número de operaciones relevantes, o primitivas, que realiza el algoritmo $\mathcal{A}$ para resolver el problema $\mathcal{P}$ con la entrada $\mathcal{I}$** .

Para el ejemplo anterior, las operaciones primitivas son toda comparación que nuestro algoritmo realiza.

Análisis de un Algoritmo

- Sea $\mathcal{P}$ un problema bien definido en nuestro modelo de computo, como el de la slide anterior. $\mathcal{P}$: encontrar el elemento menor de un arreglo de enteros $A[1..n]$.

Definimos a $D_{\mathcal{P}}(n)$ como al **conjunto de todas las posibles entradas de tamaño n al problema $\mathcal{P}$** .

Para el problema $\mathcal{P}$, ¿Cuánto es $|D_{\mathcal{P}}(n)|$?, con $n = 12$.

- Llamamos **Instancia** $\mathcal{I} \in D_{\mathcal{P}}(n)$ a una entrada específica de tamaño n , y definimos $T_{\mathcal{A},\mathcal{P}}(\mathcal{I})$ como el **número de operaciones relevantes, o primitivas, que realiza el algoritmo $\mathcal{A}$ para resolver el problema $\mathcal{P}$ con la entrada $\mathcal{I}$** .

Para el ejemplo anterior, las operaciones primitivas son toda comparación que nuestro algoritmo realiza.

Es decir: $T_{\mathcal{A},\mathcal{P}}(n) = 2n - 1$, o simplemente $T(n) = c \cdot n$, con $c > 0$ constante y sobreentendiendo quién es $\mathcal{A}$ y $\mathcal{P}$.

Análisis de un Algoritmo

- Sea $\mathcal{P}$ un problema bien definido en nuestro modelo de computo, como el de la slide anterior. $\mathcal{P}$: encontrar el elemento menor de un arreglo de enteros $A[1..n]$.

Definimos a $D_{\mathcal{P}}(n)$ como al **conjunto de todas las posibles entradas de tamaño n al problema $\mathcal{P}$** .

Para el problema $\mathcal{P}$, ¿Cuánto es $|D_{\mathcal{P}}(n)|$?, con $n = 12$.

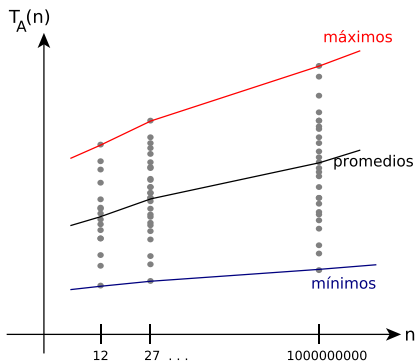
- Llamamos **Instancia** $\mathcal{I} \in D_{\mathcal{P}}(n)$ a una entrada específica de tamaño n , y definimos $T_{\mathcal{A},\mathcal{P}}(\mathcal{I})$ como el **número de operaciones relevantes, o primitivas, que realiza el algoritmo $\mathcal{A}$ para resolver el problema $\mathcal{P}$ con la entrada $\mathcal{I}$** .

Para el ejemplo anterior, las operaciones primitivas son toda comparación que nuestro algoritmo realiza.

Es decir: $T_{\mathcal{A},\mathcal{P}}(n) = 2n - 1$, o simplemente $T(n) = c \cdot n$, con $c > 0$ constante y sobreentendiendo quién es $\mathcal{A}$ y $\mathcal{P}$.

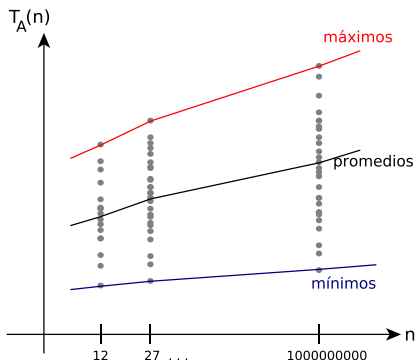
¿Qué hace que nuestro valor $T_{\mathcal{A},\mathcal{P}}(n)$ tenga sentido? es decir que tengamos la seguridad que nuestro cálculo $T(n) = 2n - 1$ es fidedigno y no varía.

Diferentes Curvas para $T(n)$



Tenemos que para diferentes entradas de un mismo n hay diferentes cantidades de operaciones primitivas.

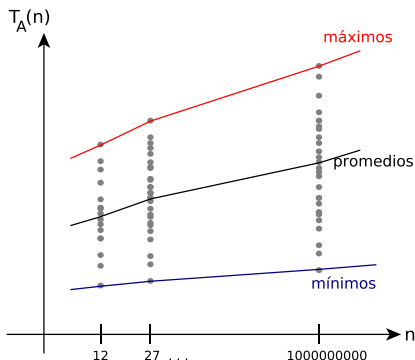
Diferentes Curvas para $T(n)$



Tenemos que para diferentes entradas de un mismo n hay diferentes cantidades de operaciones primitivas.

¿Qué nombres reciben las curvas que unen: los máximos, los promedios y los mínimos? **¿Cuál de ellas es la más importante para el análisis de $T_{\mathcal{A},\mathcal{P}}(n)$?**

Diferentes Curvas para $T(n)$



Tenemos que para diferentes entradas de un mismo n hay diferentes cantidades de operaciones primitivas.

¿Qué nombres reciben las curvas que unen: los máximos, los promedios y los mínimos? **¿Cuál de ellas es la más importante para el análisis de $T_{A,P}(n)$?**

La de los Máximos: $W_{A,P}(n)$ conocida como **función de peor caso**, o trabajo del algoritmo A :

$$W_{A,P}(n) = \max_{I \in D_P(n)} \{T_{A,P}(I)\}$$

Complejidad del Problema y del Algoritmo

Del gráfico anterior destacamos tres funciones importantes que describen el tiempo de ejecución de un algoritmo $\mathcal{A}$:

- **Función de Peor Caso** (Worst), o trabajo del Algoritmo: $W_{\mathcal{A},\mathcal{P}}(n)$.
- Función de Caso Promedio (Average): $A_{\mathcal{A},\mathcal{P}}(n)$.
- Función de Mejor Caso (Best): $B_{\mathcal{A},\mathcal{P}}(n)$.

¿Qué función es la más importante en cuanto a la **Complejidad del Problema**?

Complejidad del Problema y del Algoritmo

Del gráfico anterior destacamos tres funciones importantes que describen el tiempo de ejecución de un algoritmo $\mathcal{A}$:

- **Función de Peor Caso** (Worst), o trabajo del Algoritmo: $W_{\mathcal{A},\mathcal{P}}(n)$.
- Función de Caso Promedio (Average): $A_{\mathcal{A},\mathcal{P}}(n)$.
- Función de Mejor Caso (Best): $B_{\mathcal{A},\mathcal{P}}(n)$.

¿Qué función es la más importante en cuanto a la **Complejidad del Problema**?

Cuando estudiamos el problema modelado $\mathcal{P}$, deseamos determinar una función que acote **inferiormente el peor caso** para el problema (o una aproximación asintótica bien ajustada). Esto se conoce como **Cota inferior al problema** o **Complejidad del problema**.

Complejidad del Problema y del Algoritmo

Del gráfico anterior destacamos tres funciones importantes que describen el tiempo de ejecución de un algoritmo $\mathcal{A}$:

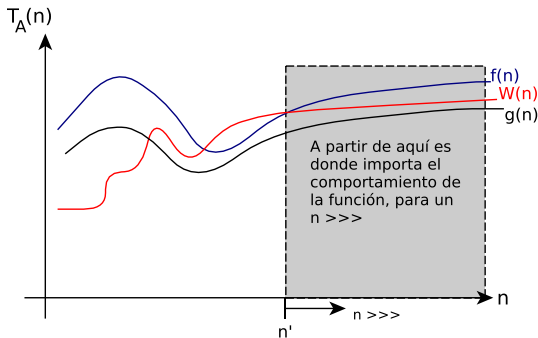
- **Función de Peor Caso** (Worst), o trabajo del Algoritmo: $W_{\mathcal{A},\mathcal{P}}(n)$.
- Función de Caso Promedio (Average): $A_{\mathcal{A},\mathcal{P}}(n)$.
- Función de Mejor Caso (Best): $B_{\mathcal{A},\mathcal{P}}(n)$.

¿Qué función es la más importante en cuanto a la **Complejidad del Problema**?

Cuando estudiamos el problema modelado $\mathcal{P}$, deseamos determinar una función que acote **inferiormente el peor caso** para el problema (o una aproximación asintótica bien ajustada). Esto se conoce como **Cota inferior al problema o Complejidad del problema**.

Cuando estudiamos un algoritmo $\mathcal{A}$ que resuelve el problema $\mathcal{P}$, deseamos determinar una función que acote **superiormente el peor caso** para $\mathcal{A}$ (o una aproximación asintótica bien ajustada). Esto se conoce como **Cota superior del algoritmo o Complejidad del algoritmo**.

Comportamiento Asintótico



Podemos aproximar $W_{A,P}(n)$ en base a otras funciones conocidas.

Suponga que la expresión matemática de la función $W_{A,P}(n)$ es compleja, pero asintóticamente acotada por una función sencilla y bien conocida; entonces podemos aproximar $W_{A,P}(n)$ a dicha función.

Ejemplo, suponga $W_{A,P}(n) = 17/3n^2 - \sqrt{131}n + 5324$, $f(n) = 50n^2$ y $g(n) = 2n^2$ entonces podemos considerar simplemente $W_{A,P}(n) = n^2$, ya que a partir de cierto $n' \gg \gg$, $\forall n \geq n'$, $W_{A,P}(n)$ será muy cercana a n^2 .

Problema de Ordenamiento

Sea $\mathcal{P}$ el problema de ordenar crecientemente una secuencia de elementos $\langle a_1, a_2, \dots, a_n \rangle$ provenientes de un universo que establece un **orden total**.

→ ¿Qué significa eso?

Problema de Ordenamiento

Sea $\mathcal{P}$ el problema de ordenar crecientemente una secuencia de elementos $\langle a_1, a_2, \dots, a_n \rangle$ provenientes de un universo que establece un **orden total**.

→ ¿Qué significa eso?

La salida es una permutación de la entrada, $\langle a'_1, a'_2, \dots, a'_n \rangle$, tq:

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

Problema de Ordenamiento

Sea $\mathcal{P}$ el problema de ordenar crecientemente una secuencia de elementos $\langle a_1, a_2, \dots, a_n \rangle$ provenientes de un universo que establece un **orden total**.

→ ¿Qué significa eso?

La salida es una permutación de la entrada, $\langle a'_1, a'_2, \dots, a'_n \rangle$, tq:

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

Solución. $\mathcal{A}$: Algoritmo Insertion Sort.

Input: cómo instancia $\mathcal{I} \in D_{\mathcal{P}}(n) \rightarrow$ un arreglo de enteros $A[1..n]$

Output: A ordenado crecientemente, tq: $A[1] \leq A[2] \leq \dots \leq A[n]$.

Problema de Ordenamiento

Sea $\mathcal{P}$ el problema de ordenar crecientemente una secuencia de elementos $\langle a_1, a_2, \dots, a_n \rangle$ provenientes de un universo que establece un **orden total**.

→ ¿Qué significa eso?

La salida es una permutación de la entrada, $\langle a'_1, a'_2, \dots, a'_n \rangle$, tq:

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

Solución. $\mathcal{A}$: Algoritmo Insertion Sort.

Input: cómo instancia $\mathcal{I} \in D_{\mathcal{P}}(n) \rightarrow$ un arreglo de enteros $A[1..n]$

Output: A ordenado crecientemente, tq: $A[1] \leq A[2] \leq \dots \leq A[n]$.

Este algoritmo:

- Opera *in place*

Problema de Ordenamiento

Sea $\mathcal{P}$ el problema de ordenar crecientemente una secuencia de elementos $\langle a_1, a_2, \dots, a_n \rangle$ provenientes de un universo que establece un **orden total**.

→ ¿Qué significa eso?

La salida es una permutación de la entrada, $\langle a'_1, a'_2, \dots, a'_n \rangle$, tq:

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

Solución. $\mathcal{A}$: Algoritmo Insertion Sort.

Input: cómo instancia $\mathcal{I} \in D_{\mathcal{P}}(n) \rightarrow$ un arreglo de enteros $A[1..n]$

Output: A ordenado crecientemente, tq: $A[1] \leq A[2] \leq \dots \leq A[n]$.

Este algoritmo:

- Opera *in place*, quiere decir que solo un número constante (y se entiende pequeño) de elementos están almacenados fuera de la estructura de entrada en cualquier instante de tiempo.

Problema de Ordenamiento

Sea $\mathcal{P}$ el problema de ordenar crecientemente una secuencia de elementos $\langle a_1, a_2, \dots, a_n \rangle$ provenientes de un universo que establece un **orden total**.

→ ¿Qué significa eso?

La salida es una permutación de la entrada, $\langle a'_1, a'_2, \dots, a'_n \rangle$, tq:

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

Solución. $\mathcal{A}$: Algoritmo Insertion Sort.

Input: cómo instancia $\mathcal{I} \in D_{\mathcal{P}}(n) \rightarrow$ un arreglo de enteros $A[1..n]$

Output: A ordenado crecientemente, tq: $A[1] \leq A[2] \leq \dots \leq A[n]$.

Este algoritmo:

- Opera *in place*, quiere decir que solo un número constante (y se entiende pequeño) de elementos están almacenados fuera de la estructura de entrada en cualquier instante de tiempo.
- Es eficiente para un n relativamente pequeño o cuando la entrada está simiordenada.

Insertion Sort

Input: un arreglo de enteros $A[1..n]$

Output: A ordenado crecientemente.

```
insertionSort( $A, n$ ){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

1	2	3	4	5	6
5	2	4	6	1	3

1	2	3	4	5	6
2	5	4	6	1	3

1	2	3	4	5	6
2	4	5	6	1	3

1	2	3	4	5	6
2	4	5	6	1	3

1	2	3	4	5	6
1	2	4	5	6	3

1	2	3	4	5	6
1	2	3	4	5	6

Insertion Sort

Input: un arreglo de enteros $A[1..n]$

Output: A ordenado crecientemente.

```
insertionSort( $A, n$ ){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

1	2	3	4	5	6
5	2	4	6	1	3

1	2	3	4	5	6
2	5	4	6	1	3

1	2	3	4	5	6
2	4	5	6	1	3

1	2	3	4	5	6
2	4	5	6	1	3

1	2	3	4	5	6
1	2	4	5	6	3

1	2	3	4	5	6
1	2	3	4	5	6

- ¿De qué depende el tiempo de ejecución de $\mathcal{A}$?

Insertion Sort

Input: un arreglo de enteros $A[1..n]$

Output: A ordenado crecientemente.

```
insertionSort( $A, n$ ){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

1	2	3	4	5	6
5	2	4	6	1	3

1	2	3	4	5	6
2	5	4	6	1	3

1	2	3	4	5	6
2	4	5	6	1	3

1	2	3	4	5	6
2	4	5	6	1	3

1	2	3	4	5	6
1	2	4	5	6	3

1	2	3	4	5	6
1	2	3	4	5	6

- ¿De qué depende el tiempo de ejecución de $\mathcal{A}$?
- Entonces, ¿Que análisis teórico podemos hacer para tener una idea de cual es el tiempo de ejecución de $\mathcal{A}$?

Insertion Sort

Input: un arreglo de enteros $A[1..n]$

Output: A ordenado crecientemente.

```
insertionSort( $A, n$ ){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

1	2	3	4	5	6
5	2	4	6	1	3

1	2	3	4	5	6
2	5	4	6	1	3

1	2	3	4	5	6
2	4	5	6	1	3

1	2	3	4	5	6
2	4	5	6	1	3

1	2	3	4	5	6
1	2	4	5	6	3

1	2	3	4	5	6
1	2	3	4	5	6

- ¿De qué depende el tiempo de ejecución de $\mathcal{A}$?
- Entonces, ¿Que análisis teórico podemos hacer para tener una idea de cual es el tiempo de ejecución de $\mathcal{A}$?
- Lo que deseamos es una buena cota superior para el algoritmo; es decir, queremos determinar $W_{\mathcal{A}, \mathcal{P}}(n) = T_{\mathcal{A}, \mathcal{P}}(n) = T(n)$, que es el número de operaciones relevantes que ejecuta $\mathcal{A}$ para una instancia cualquiera $\mathcal{I} \in D_{\mathcal{P}}(n)$, lo que nos da una relación directa con el tiempo de ejecución de $\mathcal{A}$ en el peor caso posible.

Análisis del Insertion Sort

Determinar el número exacto $T_{\mathcal{A}, \mathcal{P}}(n)$, en los 3 casos, si las operaciones primitivas son el número de comparaciones totales que se ejecutan.

Análisis del Insertion Sort

Determinar el número exacto $T_{\mathcal{A}, \mathcal{P}}(n)$, en los 3 casos, si las operaciones primitivas son el número de comparaciones totales que se ejecutan.

```
insertionSort(A, n){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Análisis del Insertion Sort

Determinar el número exacto $T_{\mathcal{A}, \mathcal{P}}(n)$, en los 3 casos, si las operaciones primitivas son el número de comparaciones totales que se ejecutan.

```
insertionSort(A, n){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Mejor Caso $B_{\mathcal{A}, \mathcal{P}}(n)$: $\rightarrow$ ¿Cuál es exactamente?

Análisis del Insertion Sort

Determinar el número exacto $T_{\mathcal{A}, \mathcal{P}}(n)$, en los 3 casos, si las operaciones primitivas son el número de comparaciones totales que se ejecutan.

```
insertionSort( $A, n$ ){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Mejor Caso $B_{\mathcal{A}, \mathcal{P}}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- Primero, note que el for tiene una comparación mientras que el while tiene un AND con 2 comparaciones que siempre se evalúan.

Análisis del Insertion Sort

Determinar el número exacto $T_{\mathcal{A}, \mathcal{P}}(n)$, en los 3 casos, si las operaciones primitivas son el número de comparaciones totales que se ejecutan.

```
insertionSort( $A, n$ ){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Mejor Caso $B_{\mathcal{A}, \mathcal{P}}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- Primero, note que el for tiene una comparación mientras que el while tiene un AND con 2 comparaciones que siempre se evalúan.
- La línea del for siempre se ejecuta exactamente n veces $\Rightarrow$ **acumulamos n operaciones**

Análisis del Insertion Sort

Determinar el número exacto $T_{\mathcal{A}, \mathcal{P}}(n)$, en los 3 casos, si las operaciones primitivas son el número de comparaciones totales que se ejecutan.

```
insertionSort( $A, n$ ){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Mejor Caso $B_{\mathcal{A}, \mathcal{P}}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- Primero, note que el for tiene una comparación mientras que el while tiene un AND con 2 comparaciones que siempre se evalúan.
- La línea del for siempre se ejecuta exactamente n veces $\Rightarrow$ **acumulamos n operaciones**
- La línea del while se ejecuta siempre 1 sola vez por cada una de las $n - 1$ iteraciones del for, ya que A está ordenado. $\Rightarrow$ **acumulamos $2(n - 1)$ op.**

Análisis del Insertion Sort

Determinar el número exacto $T_{\mathcal{A}, \mathcal{P}}(n)$, en los 3 casos, si las operaciones primitivas son el número de comparaciones totales que se ejecutan.

```
insertionSort(A, n){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Mejor Caso $B_{\mathcal{A}, \mathcal{P}}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- Primero, note que el for tiene una comparación mientras que el while tiene un AND con 2 comparaciones que siempre se evalúan.
 - La línea del for siempre se ejecuta exactamente n veces $\Rightarrow$ **acumulamos n operaciones**
 - La línea del while se ejecuta siempre 1 sola vez por cada una de las $n - 1$ iteraciones del for, ya que A está ordenado. $\Rightarrow$ **acumulamos $2(n - 1)$ op.**
- $\therefore B_{\mathcal{A}, \mathcal{P}}(n) = n + 2n - 2 = 3n - 2$

...Análisis del Insertion Sort

```
insertionSort(A, n){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Peor Caso $W_{A,P}(n)$: $\rightarrow$ ¿Cuál es exactamente?

...Análisis del Insertion Sort

```
insertionSort(A, n){  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Peor Caso $W_{A,P}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- La línea del for se ejecuta exactamente n veces $\Rightarrow$ **acumula** n

...Análisis del Insertion Sort

```
insertionSort(A, n){  
  for i = 2 to n do  
    key = A[i]  
    j = i - 1  
    while (j > 0 AND A[j] > key) do  
      A[j + 1] = A[j]  
      j = j - 1  
    A[j + 1] = key  
}
```

Peor Caso $W_{A,P}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- La línea del for se ejecuta exactamente n veces $\Rightarrow$ **acumula** n
- El while tiene un AND con 2 comparaciones que siempre se evalúan, salvo cuando $j = 0$ y sale sin evaluar la segunda condición.

...Análisis del Insertion Sort

```
insertionSort(A, n){  
  for i = 2 to n do  
    key = A[i]  
    j = i - 1  
    while (j > 0 AND A[j] > key) do  
      A[j + 1] = A[j]  
      j = j - 1  
    A[j + 1] = key  
}
```

Peor Caso $W_{A,P}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- La línea del for se ejecuta exactamente n veces $\Rightarrow$ **acumula** n
- El while tiene un AND con 2 comparaciones que siempre se evalúan, salvo cuando $j = 0$ y sale sin evaluar la segunda condición.
 - $\Rightarrow$ **acumula** $(n - 1)$ para los casos cuando $j = 0$
 - $\Rightarrow$ **acumula** $2(1 + 2 + 3.. + n - 1)$ para los otros casos cuando $j > 0$, ya que A esta ordenado decrecientemente el while se ejecuta hasta que $j = 0$.

...Análisis del Insertion Sort

```
insertionSort(A, n){  
  for i = 2 to n do  
    key = A[i]  
    j = i - 1  
    while (j > 0 AND A[j] > key) do  
      A[j + 1] = A[j]  
      j = j - 1  
    A[j + 1] = key  
}
```

Peor Caso $W_{A,P}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- La línea del for se ejecuta exactamente n veces $\Rightarrow$ **acumula** n
- El while tiene un AND con 2 comparaciones que siempre se evalúan, salvo cuando $j = 0$ y sale sin evaluar la segunda condición.
 - $\Rightarrow$ **acumula** $(n - 1)$ para los casos cuando $j = 0$
 - $\Rightarrow$ **acumula** $2(1 + 2 + 3.. + n - 1)$ para los otros casos cuando $j > 0$, ya que A esta ordenado decrecientemente el while se ejecuta hasta que $j = 0$.

$$\therefore W_{A,P}(n) = n + (n - 1) + 2 \sum_{j=1}^{n-1} j =$$

...Análisis del Insertion Sort

```
insertionSort(A, n){  
  for i = 2 to n do  
    key = A[i]  
    j = i - 1  
    while (j > 0 AND A[j] > key) do  
      A[j + 1] = A[j]  
      j = j - 1  
    A[j + 1] = key  
}
```

Peor Caso $W_{A,P}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- La línea del for se ejecuta exactamente n veces $\Rightarrow$ **acumula** n
- El while tiene un AND con 2 comparaciones que siempre se evalúan, salvo cuando $j = 0$ y sale sin evaluar la segunda condición.
 - $\Rightarrow$ **acumula** $(n - 1)$ para los casos cuando $j = 0$
 - $\Rightarrow$ **acumula** $2(1 + 2 + 3.. + n - 1)$ para los otros casos cuando $j > 0$, ya que A esta ordenado decrecientemente el while se ejecuta hasta que $j = 0$.

$$\therefore W_{A,P}(n) = n + (n - 1) + 2 \sum_{j=1}^{n-1} j = 2n - 1 + 2 \frac{n(n - 1)}{2}$$

...Análisis del Insertion Sort

```
insertionSort(A, n){  
  for i = 2 to n do  
    key = A[i]  
    j = i - 1  
    while (j > 0 AND A[j] > key) do  
      A[j + 1] = A[j]  
      j = j - 1  
    A[j + 1] = key  
}
```

Peor Caso $W_{A,P}(n)$: $\rightarrow$ ¿Cuál es exactamente?

- La línea del for se ejecuta exactamente n veces $\Rightarrow$ **acumula** n
- El while tiene un AND con 2 comparaciones que siempre se evalúan, salvo cuando $j = 0$ y sale sin evaluar la segunda condición.
 - $\Rightarrow$ **acumula** $(n - 1)$ para los casos cuando $j = 0$
 - $\Rightarrow$ **acumula** $2(1 + 2 + 3.. + n - 1)$ para los otros casos cuando $j > 0$, ya que A esta ordenado decrecientemente el while se ejecuta hasta que $j = 0$.

$$\therefore W_{A,P}(n) = n + (n - 1) + 2 \sum_{j=1}^{n-1} j = 2n - 1 + 2 \frac{n(n - 1)}{2}$$

$$= 2n - 1 + n^2 - n = n^2 + n - 1$$

...Análisis del Insertion Sort

```
insertionSort( $A, n$ ) {  
  for  $i = 2$  to  $n$  do  
     $key = A[i]$   
     $j = i - 1$   
    while ( $j > 0$  AND  $A[j] > key$ ) do  
       $A[j + 1] = A[j]$   
       $j = j - 1$   
     $A[j + 1] = key$   
}
```

Ejercicio(de tarea) determine $T_{\mathcal{A}, \mathcal{P}}(n)$ para el caso promedio.

Segmento con Loops Anidados

¿Cómo podemos analizar el tiempo exacto que toma un segmento de pseudocódigo con varios loops anidados?

```
for  $i = 1$  to  $n$  do
  for  $j = 1$  to  $2 \cdot i$  do
     $k = j$ 
    while ( $k \geq 0$ ) do
      ⟨operación primitiva (costo 1)⟩
       $k = k - 1$ 
    }
  }
}
```

Segmento con Loops Anidados

¿Cómo podemos analizar el tiempo exacto que toma un segmento de pseudocódigo con varios loops anidados?

```
for  $i = 1$  to  $n$  do
  for  $j = 1$  to  $2 \cdot i$  do
     $k = j$ 
    while ( $k \geq 0$ ) do
      ⟨operación primitiva (costo 1)⟩
       $k = k - 1$ 
    }
  }
}
```

Recuerde que:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^n i^2 = \frac{2n^3 + 3n^2 + n}{6}$$

$$\sum_{i=0}^{k-1} a^i = \frac{1 - a^k}{1 - a}$$

¿Cuanto vale $T(n)$ para el código dado?

Segmento con Loops Anidados

¿Cómo podemos analizar el tiempo exacto que toma un segmento de pseudocódigo con varios loops anidados?

```
for  $i = 1$  to  $n$  do
  for  $j = 1$  to  $2 \cdot i$  do
     $k = j$ 
    while ( $k \geq 0$ ) do
      ⟨operación primitiva (costo 1)⟩
       $k = k - 1$ 
    }
  }
}
```

Recuerde que:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^n i^2 = \frac{2n^3 + 3n^2 + n}{6}$$

$$\sum_{i=0}^{k-1} a^i = \frac{1 - a^k}{1 - a}$$

¿Cuanto vale $T(n)$ para el código dado?

$$\Rightarrow T(n) = \frac{4n^3 + 15n^2 + 11n}{6}$$

Aproximación de Sumatorias

De lo anteriormente visto, podemos notar que lo más importante es encontrar cotas superiores a las sumatorias obtenidas en nuestros análisis. En este caso muchas veces conviene aproximar la sumatoria con alguna técnica.

Aproximación de Sumatorias

De lo anteriormente visto, podemos notar que lo más importante es encontrar cotas superiores a las sumatorias obtenidas en nuestros análisis. En este caso muchas veces conviene aproximar la sumatoria con alguna técnica.

- Técnica de **Valores Brutos** como límite superior:

Consiste en reemplazar cada término en la sumatoria con una simple cota superior. Por ejemplo:

$$\sum_{i=1}^n i^2$$

Aproximación de Sumatorias

De lo anteriormente visto, podemos notar que lo más importante es encontrar cotas superiores a las sumatorias obtenidas en nuestros análisis. En este caso muchas veces conviene aproximar la sumatoria con alguna técnica.

- Técnica de **Valores Brutos** como límite superior:

Consiste en reemplazar cada término en la sumatoria con una simple cota superior. Por ejemplo:

$$\sum_{i=1}^n i^2 \leq \sum_{i=1}^n n^2$$

Aproximación de Sumatorias

De lo anteriormente visto, podemos notar que lo más importante es encontrar cotas superiores a las sumatorias obtenidas en nuestros análisis. En este caso muchas veces conviene aproximar la sumatoria con alguna técnica.

- Técnica de **Valores Brutos** como límite superior:

Consiste en reemplazar cada término en la sumatoria con una simple cota superior. Por ejemplo:

$$\sum_{i=1}^n i^2 \leq \sum_{i=1}^n n^2 = n^3$$

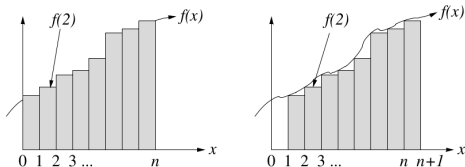
Aproximación de Sumatorias

- Aproximación por medio de Integrales:

Aproximación de Sumatorias

- Aproximación por medio de Integrales:

$$\int_0^n f(x)dx \leq \sum_{i=1}^n f(i) \leq \int_1^{n+1} f(x)dx.$$

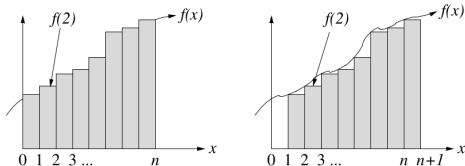


Note que el área sombreada representa exactamente nuestra sumatoria. Solo es posible aplicar esta técnica cuando $f(x)$ es una función monotona creciente. Así, la integral que representa al área bajo la curva, que se forma por el trazo de la función $f(x)$ y el eje X en el intervalo de interes; la podemos utilizar tanto para una cota inferior como superior de nuestra sumatoria.

Aproximación de Sumatorias

- Aproximación por medio de Integrales:

$$\int_0^n f(x)dx \leq \sum_{i=1}^n f(i) \leq \int_1^{n+1} f(x)dx.$$



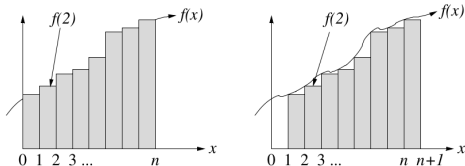
Note que el área sombreada representa exactamente nuestra sumatoria. Solo es posible aplicar esta técnica cuando $f(x)$ es una función monotonamente creciente. Así, la integral que representa al área bajo la curva, que se forma por el trazo de la función $f(x)$ y el eje X en el intervalo de interés; la podemos utilizar tanto para una cota inferior como superior de nuestra sumatoria. Por ejemplo:

$$\sum_{i=1}^n i^2$$

Aproximación de Sumatorias

- Aproximación por medio de Integrales:

$$\int_0^n f(x)dx \leq \sum_{i=1}^n f(i) \leq \int_1^{n+1} f(x)dx.$$



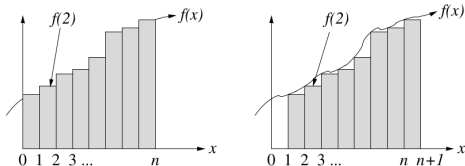
Note que el área sombreada representa exactamente nuestra sumatoria. Solo es posible aplicar esta técnica cuando $f(x)$ es una función monótona creciente. Así, la integral que representa al área bajo la curva, que se forma por el trazo de la función $f(x)$ y el eje X en el intervalo de interés; la podemos utilizar tanto para una cota inferior como superior de nuestra sumatoria. Por ejemplo:

$$\sum_{i=1}^n i^2 \leq \int_1^{n+1} x^2 dx$$

Aproximación de Sumatorias

- Aproximación por medio de Integrales:

$$\int_0^n f(x)dx \leq \sum_{i=1}^n f(i) \leq \int_1^{n+1} f(x)dx.$$



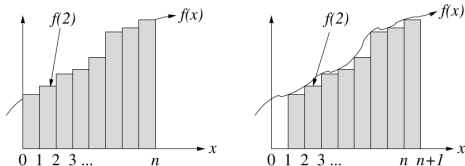
Note que el área sombreada representa exactamente nuestra sumatoria. Solo es posible aplicar esta técnica cuando $f(x)$ es una función monótona creciente. Así, la integral que representa al área bajo la curva, que se forma por el trazo de la función $f(x)$ y el eje X en el intervalo de interés; la podemos utilizar tanto para una cota inferior como superior de nuestra sumatoria. Por ejemplo:

$$\sum_{i=1}^n i^2 \leq \int_1^{n+1} x^2 dx = \left. \frac{x^3}{3} \right|_{x=1}^{n+1}$$

Aproximación de Sumatorias

- Aproximación por medio de Integrales:

$$\int_0^n f(x)dx \leq \sum_{i=1}^n f(i) \leq \int_1^{n+1} f(x)dx.$$



Note que el área sombreada representa exactamente nuestra sumatoria. Solo es posible aplicar esta técnica cuando $f(x)$ es una función monotona creciente. Así, la integral que representa al área bajo la curva, que se forma por el trazo de la función $f(x)$ y el eje X en el intervalo de interes; la podemos utilizar tanto para una cota inferior como superior de nuestra sumatoria. Por ejemplo:

$$\sum_{i=1}^n i^2 \leq \int_1^{n+1} x^2 dx = \left. \frac{x^3}{3} \right|_{x=1}^{n+1} = \frac{(n+1)^3}{3} - \frac{1}{3} = \frac{n^3 + 3n^2 + 3n}{3}$$